

Como Go ajudou a construir a

Fila de Aposentadoria

Sobre Mim

Felipe Felix Souza dos Santos

- Servidor de carreira da SEPLAG;
- Atuando na área da tecnologia desde de 2022;
- **2022 – 2025:** Desenvolvimento de sistemas no DEPEN usando PHP (Laravel) e JavaScript;
- **2025 – atualmente:** Desenvolvimento de APIs e automações no Centro de Automatização e Inteligência Artificial – Automatiza.MG;

Situação Problema



Criação de uma fila de aposentadoria com base em critérios objetivos

- Mais de 3.000 processos de aposentadoria aguardando análise;
- Cada processo possui dezenas de documentos;
- Distinção entre processos de pedido de aposentadoria e demais processos;
- Cálculo de um “score” com base em dados do servidor, como: idade, data do pedido, se possui alguma deficiência, etc;
- Atribuição automática dos processos aos analistas.

O que precisávamos fazer

1260.01.00XXXXXX/XXXX-XX



Informações		
Status	Score	Prioritário
Análise Pendente	1,00	Não
Analista Responsável	CPF Requerente	Data de Nascimento
Não possui	597. █████ -53	09/03/1966 (60 anos)
Unidade de Origem	Data Requerimento	Judicial
SEE/SREA - Apos. e Benef.	03/04/2024	Não
Invalidez		
Não		

Nosso Desafio



O grande desafio da Fila de Aposentadoria é analisar um processo SEI, o que envolve as seguintes etapas:

1. Buscar os dados do processo na API do SEI;
2. Listar os documentos do processo (*webscraping* do link externo);
3. Analisar cada documento individualmente (N vezes):
 1. Baixar o conteúdo do documento;
 2. Fazer o hash para evitar o reprocessamento;
 3. Conferir se o documento ainda não foi analisado;
 4. Buscar os metadados na API do SEI (assinaturas, tipo, etc.);
 5. Extrair o texto (OCR ou converter HTML para Markdown);
 6. Analisar o documento com IA (resumo, dados principais);
4. Com o contexto dos documentos, analisar com IA se o processo é de aposentadoria e extrair os dados do requerente;

Por que Go?



O motivo principal é **concorrência**, permitindo que os N documentos de um processo sejam analisados “ao mesmo tempo”, reduzindo **significativamente** o tempo de análise dos processos.

Exemplo:

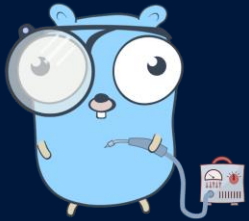
```
documentos := []string{"100001", "100002", "100003"}

// Cria um grupo de espera.
var wg sync.WaitGroup

// Faz um loop nos documentos.
for _, documento := range documentos {
    // Inicia o processamento do documento em background.
    // Processa os 3 documentos "de uma vez".
    wg.Go(func() {
        // Processa o documento.
        err := processarDocumento(documento)
        // Verifica se houve erro no processamento e gera um log.
        if err != nil {
            log.Printf("Não foi possível analisar o documento %s: %v", documento, err)
        }
    })
}

// Aguarda o processamento dos documentos finalizarem.
wg.Wait()
```

Outras Vantagens



Além da **concorrência**, a linguagem possui outras vantagens:

1. **Simplicidade (Código Limpo):** Apenas 25 *keywords*. Código padronizado e fácil de ler e de aprender.
2. **Errors como valores:** Sem blocos *try/catch* que escondem falhas. O tratamento é explícito e imediato, tornando o comportamento do sistema previsível e muito mais seguro.
3. **Tipagem Estática:** Erros são pegos na compilação, antes de ir para produção. Garante refatorações seguras e contratos claros entre componentes, essencial para sistemas robustos e de grande escala.

Muito Obrigado!

